The oscillator ladder algebra: $[b, b^\dagger] = 1$

Claude, with Daniel Murfet

2026-09-06

Abstract

Unit 16 of the grammar-paper §4 autoformalisation: the oscillator ladder algebra `lem:oscillator_algebra`. With $b^\dagger = \beta^{-1/2}\partial_a$ and $b = 2\beta^{-1/2}\partial_a - \beta^{1/2}a$ as operators on functions of a, we prove the canonical commutator $[b, b^\dagger] = 1$ (the Weyl relation) and identify the ladder actions on the fluctuation function, $b^\dagger S_\lambda = \beta^{1/2} S_{\lambda+1/2}$ and $b\, S_\lambda = (2\lambda - 1)\beta^{-1/2} S_{\lambda-1/2}$. Zero `sorry`/`axiom`.

1 Setting

The QHO ladder was promised throughout §4: the derivative ladder $S'_\lambda = \beta S_{\lambda+1/2}$ (unit 2) and the lowering identity $2S'_\lambda - \beta a S_\lambda = (2\lambda - 1)S_{\lambda-1/2}$ (unit 5) are the raising and lowering *actions*, but the *operators* $b, b^\dagger$ and their commutator had not been formalised. This unit supplies them, closing `lem:oscillator_algebra`.

2 The things formalised

```
noncomputable def raiseOp ( : ) (f :  → ) :  →  := fun a =>  ^ (-(1: )/2) * deriv f a
noncomputable def lowerOp ( : ) (f :  → ) :  →  :=
  fun a => 2 *  ^ (-(1: )/2) * deriv f a -  ^ ((1: )/2) * (a * f a)

theorem ladder_commutator ( : ) (h : 0 < ) (f :  → )
    (hf1 : Differentiable  f) (hf2 : Differentiable  (deriv f)) (a : ) :
    lowerOp  (raiseOp  f) a - raiseOp  (lowerOp  f) a = f a

theorem raiseOp_fluctuation ... :
    raiseOp  (fun a => fluctuation  lam a) a = ^((1: )/2) * fluctuation  (lam + 1/2) a
theorem lowerOp_fluctuation ... (hlam : 1/2 < lam) :
    lowerOp  (fun a => fluctuation  lam a) a
      = (2*lam - 1) * ^(-(1: )/2) * fluctuation  (lam - 1/2) a
```

3 Proof strategy

Commutator. Compute $b(b^\dagger f) = 2\beta^{-1}f'' - af'$ and $b^\dagger(bf) = 2\beta^{-1}f'' - f - af'$; the difference is f. In Lean, the two second-derivative values are obtained via `HasDerivAt` (`const_mul`, `mul` of `hasDerivAt_id` with f, `sub`), converted to `deriv` by `.deriv`; then a single `linear_combination (f a) * hr2` over $\beta^{1/2}\beta^{-1/2} = 1$ finishes — the $2\beta^{-1}f''$ and af' terms cancel by `ring`, leaving the $\beta^{1/2}\beta^{-1/2} = 1$ factor.

Ladder actions. $b^\dagger S_\lambda = \beta^{-1/2} S'_\lambda = \beta^{-1/2}\beta S_{\lambda+1/2} = \beta^{1/2} S_{\lambda+1/2}$ (`deriv_fluctuation` plus $\beta^{-1/2}\beta = \beta^{1/2}$). $b\, S_\lambda = \beta^{-1/2}(2S'_\lambda - \beta a S_\lambda) = \beta^{-1/2}(2\lambda - 1) S_{\lambda-1/2}$ (`fluctuation_lowering`, after factoring $\beta^{-1/2}$ out of $2\beta^{-1/2} S' - \beta^{1/2} a S$).

4 Roadblocks and resolutions

- `deriv_sub`/`deriv_mul` match `Pi.sub`/`Pi.mul` of two lambdas, not a single `fun x => f x - g x`; building the derivative with `HasDerivAt` combinators and finishing by `.deriv` sidesteps the pattern mismatch entirely (cleaner than the `deriv_fun_*` variants here).
- The combinator value of `(hasDerivAt_id a).mul …` is $1 \cdot fa + a \cdot f'$; keeping the literal $1 \cdot fa$ (rather than normalising with `congr_deriv`) and letting the final `linear_combination`/`ring` absorb it avoided a spurious `ring` failure.
- $\beta^{-1/2} \cdot \beta = \beta^{1/2}$: rewriting $\beta \to \beta^1$ hits the *base* of $\beta^{-1/2}$ too, so `nth_rewrite 2` targets the standalone β before `Real.rpow_add`.

5 What was Mathlib, what was new

Mathlib supplied `HasDerivAt.const_mul`/`.mul`/`.sub`/`.deriv`, `hasDerivAt_id`, and the rpow arithmetic. New: the operator definitions, the commutator (a Weyl relation on a concrete SLT family), and the identification of the ladder actions with the already-proved derivative ladder and lowering identity.

6 Lessons

For operator identities involving `deriv` of a hand-written λ with $\pm$ and products, build the derivative as a `HasDerivAt` term and take `.deriv` at the end — this avoids every `Pi.add`/`Pi.sub` pattern-match pitfall of the `deriv_*` rewrite lemmas, and lets one `linear_combination` over the scalar relation finish.

7 Follow-ups

This closes the operator side of §4's ladder. The last outstanding §4 item is `lem:log_generating` ($\sum \frac{z^p}{p!}(c - \partial_\nu)^p S_\nu = e^{zc} S_{\nu - z}$), a sum/integral interchange over `log_shift` needing the real exp series and dominated convergence (convergent for small z). The number operator and the extension of S to $\lambda \le 0$ (`defn:S_negative`) build directly on these ladder operators.